Turing Machines

Math 252

The Turing Machine Model

Motivation

The Turing machine model pre-dates electronic computers and comes from the work of Hilbert (early 1900's) and Turing and Church (1930)'s.

- Not trying to model (electronic) computers, but computation or algorithm
- Identified three essential ingredients: _____, _____, _____
  - implicitly there is a fourth ingredient: _____
- The Turing machine is a specific formalization of these basic ingredients

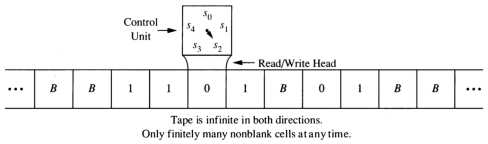
Definition of a 1-tape Turing Machine

A Turing machine consists of the following:

- input alphabet: A finite set Σ that does not include the "blank symbol"
- tape alphabet: A finite set Γ with $\Sigma \subsetneq \Gamma$; Γ includes the "blank symbol"
- states: A finite set Q
- initial state: $q_0 \in Q$
- final states: a subset of Q
- program: A partial function $\delta : \Gamma \times Q \to \Gamma \times Q \times \{L, R, S\}$.
  - A partial funciton is not required to have a value for every input. You could think of this as a function $\delta : \Gamma \times Q \to (\Gamma \times Q \times \{L, R, S\}) \cup \text{null})$, where null is some symbol not in $\Gamma \times Q \times \{L, R, S\}$.
  - A partial function can be represented as a subset of $\Gamma \times Q \times \Gamma \times Q \times \{L, R, S\}$ (This is how we will formalize it below.)

Execution of a Turing Machine program

Imagine a machine with a 2-way infinite tape divided into cells. Each cell contains exactly one symbol from the tape alphabet. At the start of the execution, the input is written on the tape, the read/write head is located at the left most symbol of the input, and all cells that don't contain part of the input contain the blank sybmol.



At each step in the execution of a Turing machine program, the read/write head is located at one cell of the tape. The program uses the current state and contents of the tape at that position to determine its action:

- write a symbol in the current cell (overwriting what was there before),
- move the head left or right (or stay put)
- update the state

This is repeated until one of two things happens:

- there is no instruction to execute, or
- the state is a final state

When either of these two things happens, the machine **halts**.

An ASCII Representation of a Turing Machine

We will be working with a particular ASCII representation of a Turing machine.

The format for a Turing machine file is the following. The first line indicates that this is A Turing machine file. The file format allows for several variations on the basic 1-tape Turing machine, but for now we will focus on the basic version.

There should be no blank lines in the file

```
Line 1: ATM.
Line 2: Name/description of the machine.
Line 3: all characters in the input alphabet, separated by spaces
Line 4: all characters in the tape alphabet, separated by spaces
Line 5: the number of tapes (N). Use 1 for a 1-tape TM.
Line 6: the number of tracks on tape 0. Use 1.
Line 8: 1 or 2, indicating tape 0 is 1-way or 2-way infinite tape
Line 9: the initial state of the Turing machine
Line 10: all the final states separated by spaces
Line 11: first of a list of transitions (see below)
Line 12: another transition
...
2nd last line: last of the list of transitions
Last line: end  // You must type the word "end" but it can be upper or lower case
```

The format of a transition is:

```
<state> <content of tape> <next state> <next content of tape> <head-move direction>
```

where

- the `<state>` and `<next state>` are the names of states and can be any strings.
- the `<content of tape>` and `<next content of tape>` are symbols from the tape alphabet.
- the `<head-move direction>` is one of R, L, or S, indicating a right or left movement or staying stationary.
- The **blank symbol** is denoted by `B` and this symbol must not be used as part of the input alphabet.
- comments can be added with `//`

Example 1

```
ATM
Example Turing Machine
0 1 // input alphabet, note such comments are permitted at the end of the line
0 1 B // tape alphabet
1 // number of tapes
1 // number of tracks on tape
2 // tape is 2-way infinite
s0 // the initial state
s3 // final state
s0 0 s0 0 R
s0 1 s1 1 R
s0 B s3 B R
s1 0 s0 0 R
s1 1 s2 1 L
s1 B s3 B R
s2 1 s3 0 R
end // required
```

1. For each input string below, determine the configuration (tape contents, location of read-write head, and state) of the Turing machine when it halts.

    a. λ
    b. 01
    c. 0101
    d. 010110
    e. 001

Example 2

```
ATM
Example Turing Machine
0 1 // input alphabet, note such comments are permitted at the end of the line
0 1 B // tape alphabet
1 // number of tapes
1 // number of tracks tape
2 // tape is 2-way infinite
[init] // the initial state
[final] // final state
[init] 0 [10] 0 R
[init] 1 [01] 0 R
[init] B [final] B S
[00] 0 [10] 0 R
[00] 1 [01] 1 R
[00] B [final] B S
[01] 0 [11] 0 R
[01] 1 [00] 1 R
[01] B [01] B R
[10] 0 [00] 0 R
[10] 1 [01] 1 R
[11] 0 [01] 0 R
[11] 1 [10] 1 R
end // required
```

2. For each input string below, determine the configuration (tape contents, location of read-write head, and state) of the Turing machine when it halts.

   a. λ
   b. 0110
   c. 01111
   d. 01110

The language recognized by a Turing Machine

We will say that a Turing machine M accepts a string x (by state) if on input x it halts in a final state. The language of a Turing machine is

$$L(M) = \{x \mid M \text{ accepts } x\}$$

3. What langauges do the previous two machines accept?

Designing a Turing Machine

4. Design a Turing machine that recognizes $(\mathbf{0} \cup \mathbf{1})\mathbf{1}(\mathbf{0} \cup \mathbf{1})^*$
5. Design a Turing machine that recognizes $\{0^n 1^n \mid n \geq 0\}$. This shows that Turing machines can recognize languages that are not regular.

Turing machines can also compute functions. The output of a Turing machine is the tape contents when it halts.

6. Design a Turing machine that adds in unary. Use the input alphabet $\{1, +\}$.

   In unary, a non-negative integer n is represented by $n + 1$ 1's.

   Example: Our machine should convert `111+1111` into _____.

A Turing machine computes **neatly** if when it halts (a) the read/write head is on the left-most non-blank tape symbol (or any blank symbol if the tape is all blanks), and (b) there are no internal blanks (ie, all the non-blank symbols are contiguous on the tape).

7. Modify your previous machine so that it computes neatly.

Variations on the Turing Machine

There are many variations on the Turing machine. Here are some examples.

- Instead of having a single tape, a Turing machine can have multiple tapes.
- Instead of being 2-way infinite, the tape(s) can have a left end. These are called 1-way infinite tapes. (Attemping to move left from the left-most cell causes the machine to halt.)
- Turing machines may be required to move the head at each step (so "stay" is not an option).
- Multi-track tapes aren't really a variation, but a handy way of thinking about a tape alphabet. (Our simulator makes this easy to do.) Since the tape alphabet may be anything, if we want to simulate writing two symbols from A, we can use a tape alphabet of $A \times A$ or even $A \cup (A \times A)$. One element of $A \times A$ represents two elements from A (one on the "top track" and one on the "bottom track").
- Instead of having a 1-diminsional tape, a tape can be a 2-dimensional grid of cells. The read/write head can move up and down in addition to left and right.

See the Turing machine specification for our simulator to learn how to write Turing machine programs that take advantage of these additional features.

Each of these variations is equivalent to the standard Turing machine with one 2-way infinite tape. So are many other models of computation. This has led to the so-called **Church-Turing Thesis**. Here is one statement of that thesis:

> A function on the natural numbers is computable by a human being following an algorithm, ignoring resource limitations, if and only if it is computable by a Turing machine.

8. Give a high level description of how to convert a Turing machine with 2-way infinite tape into a Turing machine with a 1-way infinite tape.
9. Give a high level description of how to convert a 2-tape machine into a 1-tape machine.
10. **Nested Parens** Write a Turing machine recognizes all strings that use the input alphabet $\{(,), *\}$ and the parentheses are properly nested. (The asterisks may be anywhere.)
    a. First try this with a two-tape machine.
    b. Then try this with one two-track tape.
    c. For an extra challenge, try doing it with a one-tape machine with tape alphabet $\{(,), *, \#\}$

Running time of a Turing machine

The running time of a Turing machine on input x is the number of steps before it halts. The conversions above typically change the running time, but machines that run in polynomial time are converted into machines that run in polynomial time (typically with a different degree polynomial).

P and NP

This provides a formal definition of the famous set P. P is the set of all languages that can be recognized by a Turing machine in polynomial time.

A **non-deterministic Turing machine** can have multiple transitions from the same state/tape contents configuration. A non-deterministic TM accepts an input x if there is a way of choosing transitions that causes the machine to halt in a final state. (This is similar to how non-detrministic automata were defined). NP is the set of all languages that can be recognized by a non-detrministic Turing machine.

The "P vs. NP question" is whether P and NP are the same or different. To date, this question is unresolved.